Scotty: A Compact One-Pass Decision Model Trained with Beam-Generated Data

Oliver Kramer

Carl von Ossietzky University of Oldenburg
Department of Computing Science, Germany
oliver.kramer@uni-oldenburg.de

Abstract. Scotty is a compact, one-pass decision model trained with Beam-generated data for software tasks that need a choice rather than generated prose. It reads a situation, question and candidate answers, then scores the candidates jointly in one forward pass. The 421-million-parameter implementation combines a pretrained ModernBERT-large encoder with a fresh Laya-format head. Training balances Beam-generated hard targets and Typed Decisions soft targets 50:50. Holding architecture, examples, optimization and update budget fixed, 50:50 source sampling reaches a three-seed development objective of 0.859532 ± 0.004603, versus 0.841199 ± 0.009365 for data-proportional sampling (mean $\pm$ sample SD). On a one-time group-safe source audit, seed-0 scores are 0.841031 and 0.804860. Equal sampling raises Typed hard and Beam raw accuracy by 8.13 and 0.82 points, while Beam position-balanced accuracy falls by 0.89 points. The audit uses the same source collections as training, not an external distribution, and retained aggregates do not support paired uncertainty. We release the model, recipe and artifacts.

1 Introduction

A parcel-release service, a support router and a maintenance dashboard repeatedly ask the same kind of question: given the facts, which answer from a known list should be selected? A generative model can answer, but an application must then parse the text and reject invalid outputs. A decision model instead returns one score per allowed answer.

Scotty is a compact, one-pass decision model trained with Beam-generated data for this constrained interface. It supports Boolean decisions, named categories and ordered ratings using options supplied with each request. No explanation or chain of thought is generated. Training balances the Beam examples with Typed Decisions examples 50:50; Beam contributes supervision, not weights.

Our contribution is not a new Transformer block. It is a controlled and reproducible recipe for mixing teacher-generated hard decisions with a smaller public soft-target collection. We (i) adapt a joint candidate-scoring encoder to these sources; (ii) isolate source sampling under a matched architecture and training budget; and (iii) report both three-seed development evidence and a single, previously locked source audit, including the metric that regresses.

1.1 Related work

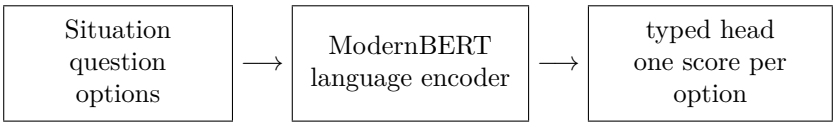
ModernBERT updates encoder-only Transformers for long-context fine-tuning and inference [3]. Our option head follows Laya's typed-decision interface [4]; we study the training recipe, not claim that architecture as new. Teacher-generated instruction data can transfer task coverage [5], and distillation transfers teacher behavior to smaller models [6]. Data-mixture work shows that sampling weights matter even when examples are unchanged [7]. We test the simplest intervention: equal source presentation for structured decisions.

2 From text to one constrained choice

Every request has three parts. The *situation* supplies relevant facts, the *question* says what must be decided, and the *options* define the answers the application can use.

> **Situation:** A parcel is sealed and labelled, but its final weight has not been recorded. The release rule requires all three checks.
>
> **Question:** Is the parcel ready for release?
>
> **Options:** No; yes.
>
> The rule supports **no** because one required check is missing. This example explains the task; it is not a measured model prediction.

The request is serialized as one sequence. The question and every candidate are placed before the situation, and each candidate receives a special `[MASK]` marker. ModernBERT reads the sequence jointly, so each option representation depends on the situation, question and competing answers.



$\longrightarrow$ softmax over supplied options $\longrightarrow$ selected answer

The fresh head adds a learned embedding for three decision types, applies two Transformer layers and uses one shared scalar scorer at option markers. Softmax normalizes the logits and the largest selects the answer. Without calibration, these are comparative scores, not verified probabilities of correctness [8].

Candidates are scored together, not in independent chunks. Full inputs are limited to 1,024 tokens. The question and option region has a 256-token budget, and each option is capped at 48 tokens. No rationale is generated. The model has 421,293,827 parameters: a pretrained `answerdotai/ModernBERT-large` encoder plus a newly initialized Laya-format head. No pretrained Laya or Beam weights are loaded.

Options may change between requests while output remains structured. Joint encoding compares alternatives directly, but cannot repair ambiguous questions, missing facts or coverage gaps.

3 Data and matched training

3.1 Two supervision sources

Reflection Beam 1.0 501B-A23B generated operational situations, questions, candidates and hard targets [1]. Checks reject malformed rows, invalid targets and detected duplicates; similarity components remain grouped across splits. API generation stayed within the $25 participant credit budget, excluding local training and hardware.

Typed Decisions covers customer service, invoices, security and agent traces [2]. Its targets may distribute support across options. Cross-entropy preserves these soft targets; Beam choices become one-hot targets. For target distribution q and predicted option scores p, every example minimizes $-\sum_i q_i \log p_i$. A hard Beam label puts all mass on one option, whereas a Typed target can reward a plausible neighboring rating without treating it as fully correct. Both forms therefore train the same head and output interface.

Before the matched comparison, a stable group-safe audit was removed from the previously used training sources. The fitting pool contains 11,948 Beam and 4,320 Typed decisions. Existing development sets contain 3,327 Beam and 1,200 Typed decisions. The locked audit contains 1,344 Beam decisions and 480 Typed decisions from 96 cases. Historical Beam and official Typed tests are not used for training, selection or this comparison.

Source	Fit	Development	Locked audit
Beam-generated	11,948	3,327	1,344
Typed Decisions	4,320	1,200	480

Table 1: Decision counts used in the controlled comparison.

3.2 The controlled sampling comparison

Both recipes start from the same pinned ModernBERT encoder and a fresh, seed-matched decision head. They use identical fitting examples, topology, objective, optimizer and compute budget. Only the sampler changes. The matched baseline cycles uniformly through the pooled fitting items; its natural source share is 73.44% Beam and 26.56% Typed. Scotty draws half of each effective batch uniformly from Beam and half uniformly from Typed, with replacement within source. This is source balance, not class balance, and it does not imply equal token counts or exactly matched floating-point operations.

The recipes differ only in source frequency. Both see every item, but 50:50 sampling revisits Typed more often. Labels, parameters and updates stay fixed.

Each run uses 849 optimizer updates, micro-batches of 8, gradient accumulation over 8 micro-batches, effective batch 64 and 54,336 presentations. AdamW updates the full encoder and head with encoder learning rate 2.5×10^{-5}, head

rate 10^{-4}, cosine decay without warmup to 10^{-6}, weight decay 0.01 and gradient clipping at 1.0. Training uses BF16 autocast, gradient checkpointing and a single NVIDIA GB10 with PyTorch 2.14.1+cu130. Checkpoints at updates 283, 566 and 849 are compared by the development objective in Section 4; the earlier update wins an exact tie.

4 Experimental evaluation

4.1 Metrics and protocol

Typed hard accuracy is the fraction of top choices matching the target's top option. This reduces a soft target to its most-supported option for evaluation; training still uses the complete distribution. Beam raw accuracy is defined analogously. Beam position-balanced accuracy first groups examples by decision type, number of options and target position, then gives every observed group equal weight. Raw accuracy asks how many decisions are correct, while the balanced metric prevents a frequent option layout from dominating. The development objective is

$$S = \frac{1}{2}(\text{Typed hard accuracy}) + \frac{1}{2}(\text{Beam position-balanced accuracy}).$$

Equal weighting prevents the larger source from dominating selection. It is a project choice, not a universal utility function; applications may value the components differently. Beam raw accuracy is shown as an interpretable decision count but is not part of S.

Three seed-matched runs use the same initialization rule and checkpoint schedule. Mean and sample SD describe the three observations, not a confidence interval. The audit uses one preselected deployment seed. Related Beam rows and duplicate components, and all Typed questions from one case, remain on one split side.

4.2 Development comparison

Table 2 reports three independent runs. Equal source sampling raises mean S by 0.018334, mainly through a 3.31-point increase in Typed accuracy; mean Beam position-balanced accuracy increases by 0.36 points. The sample standard deviations summarize only three observed seeds and are not confidence intervals. These values assess the sampling rule on development data, where the recipe and checkpoint are selected; they are not a final independent test.

The 50:50 objective is 1.83 points higher. The larger change is the 3.31-point Typed improvement; Beam balanced accuracy changes by 0.36 points. More exposure to the smaller source improves its component without reducing mean Beam balance on development. This establishes an advantage over the matched proportional reference, not global optimality.

Development recipe	S mean±SD	Typed hard	Beam bal.
Data-proportional	0.841199 ± 0.009365	0.765556	0.916842
Equal 50:50 (Scotty)	$\mathbf{0.859532} \pm 0.004603$	**0.798611**	**0.920454**

Table 2: Matched three-seed development results. SD is the sample standard deviation of S.

4.3 Locked audit and interpretation

Deployment seed 0 was fixed before the audit was opened; there was no seed selection on audit values and no post-audit adaptation.

Locked source audit	Proportional	Scotty	Difference
Objective S	0.804860	**0.841031**	+0.036171
Typed hard	0.685417	**0.766667**	+0.081250
Beam raw	0.877976	**0.886161**	+0.008185
Beam position-balanced	**0.924304**	0.915395	−0.008909

Table 3: Seed-0 matched comparison on the one-time group-safe audit.

On this audit, equal sampling improves S, Typed accuracy (39 additional correct decisions) and Beam raw accuracy (11 additional correct decisions). However, Beam position-balanced accuracy falls by 0.89 points. The method therefore trades a small amount of balance across Beam position cells for a larger gain on the under-presented Typed source; it is not uniformly superior.

Raw and balanced Beam accuracy move in opposite directions: 11 additional decisions are correct, yet the equal-weighted cell average decreases. Gains are therefore uneven across decision-type, option-count and target-position groups. Aggregates do not identify the changed cells, so both metrics are reported rather than adding a causal claim.

The matched design shows that presentation frequency changes the coverage tradeoff for this architecture, data and budget. It does not isolate teacher quality, compare source-only training or establish transfer to another model. Sampling adds no parameters or inference steps.

4.4 Evidence boundary and limitations

The audit was newly locked but drawn from the training sources, not an external distribution. Retained aggregates cannot reconstruct a case-grouped paired interval, so we make no significance claim. Synthetic supervision can transmit teacher errors; filtering was assistant-assisted, not expert annotation.

Two sources, one topology and three seeds are a narrow evidence base. Equal sampling need not suit every mixture, and no real-world evaluation is reported. Table 3 shows why the aggregate needs its components. Scores are uncalibrated; licenses, privacy and domain harms need separate review.

5 Conclusion

Scotty turns an open-ended language problem into a constrained decision by scoring a situation, question and supplied options in one pass. It learns from Beam hard choices and Typed Decisions soft targets. The contribution is a controlled source-presentation comparison with model, examples and budget fixed.

Mean three-seed S rises from 0.841199 to 0.859532 with 50:50 sampling. The audit improves combined and Typed scores, but Beam balanced accuracy falls 0.89 points. Exposure to the smaller source improves coverage while some balance across Beam layouts is lost; component metrics must accompany the aggregate.

Explicit options can simplify routing, release checks and ordered ratings, but scores remain uncalibrated and are not explanations. Deployment needs clear questions, truncation checks and domain validation. External data, more seeds, group-level predictions and learned mixture weights are the next tests.

Appendix: Reproducibility note

The public model, tokenizer, configuration and inference code are at `https://huggingface.co/DarwinGuru/scotty`. Section 3 records the training settings and split counts for the released system.

References

[1] Reflection AI. *Beam model card*, 2026. `https://reflection.ai/beam-model-card`.

[2] LocalLLaMA. Typed Decisions dataset, revision `468b1461d59e01d6404e1cf37431da962acd43a0`, 2026. `https://huggingface.co/datasets/LocalLLaMA/typed-decisions`.

[3] B. Warner et al. Smarter, better, faster, longer: A modern bidirectional encoder. *arXiv:2412.13663*, 2024.

[4] Laya contributors. *Laya: Typed decision model and implementation.* `https://github.com/NandhaKishorM/laya`, accessed 2026.

[5] Y. Wang et al. Self-Instruct: Aligning language models with self-generated instructions. *Proc. ACL*, 2023.

[6] G. Hinton et al. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[7] S. M. Xie et al. DoReMi: Optimizing data mixtures speeds up language model pretraining. *NeurIPS*, 36, 2023.

[8] C. Guo et al. On calibration of modern neural networks. *Proc. ICML*, 2017.